Q1

Consider the following job script:

```
#!/bin/bash
#BSUB -J simulate
#BSUB -q hpc
#BSUB -W 10:00
#BSUB -R "rusage[mem=???GB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o sim_%J.out
#BSUB -e sim_%J.err

python simulate.py initconds.npy
```

For the simulate.py script to run, it will need at least 100GB of memory. To achieve this, what must we insert instead of "???" in the line `#BSUB -R "rusage[mem=???GB]"`?

A) 25GB
B) 100GB
C) 10GB
D) 50GB

Correct: A)

The job script requests 4 cores, so each core must request 100 GB /4 = 25 GB.

Q2

Recall that 16-bit floating point numbers, as returned by NumPy's "finfo", have a resolution of 0.001, a minimum value of -6.55040e04 and a maximum value of 6.55040e04. Given this, what value will the following Python code print:

```
import numpy as np
a = np.array(10000, dtype='float16')
b = np.array(1, dtype='float16')
print(a + b)
```

A) 10001
B) 10000
C) 9999
D) 10000.5

Correct: B)

*With infinite precision: 10000 + 1 = 10001. The resolution of float16 is 0.001. Since 10000 * 0.001 = 10, we cannot represent 10001 = 1.0001e4, since the significant does not have enough digits. Therefore, the answer is rounded to 10000 = 1e4.*

Q3

Can the operation of set intersection ($\cap$) be used in a parallel reduction framework? Recall that the intersection of two sets is a new set containing all elements present in *both* input sets. For example, {1, 2, 3, 4} $\cap$ {2, 3, 5, 6} = {2, 3}.

A) Yes
B) No, $\cap$ *is not* associative (but it *is* commutative)
C) No, $\cap$ *is not* commutative (but it *is* associative)
D) No, $\cap$ *is neither* associative nor commutative.

Correct: A)

Set intersection is commutative, since the order doesn't matter. If x is in A and B it is in A $\cap$ B and B $\cap$ A.

Set intersection is also associative. An element, x, is in (A $\cap$ B) $\cap$ C and A $\cap$ (B $\cap$ C) if and only if it is in all sets A, B and C so the order of the intersections does not matter.

Q4

Given 2D row-wise array, `a`:

1	5	43	51	32
73	2	4	67	37
9	3	54	8	22

What is the value of `a.reshape(-1)[8]`?

A) 67
B) 51
C) 32
D) 8

Correct: A)

a.reshape(-1) reshapes the array to a 1D array, [1, 5, 43, 51, 32, 73, 2, 4, 67, 37, 9, 3, 54, 8, 22], since a is stored row-wise. The element with index 8 is then 67.

Q5

Assume you have an array of RGB images stored as an N x H x W x 3 array named “images”. For each image, you have a mean RGB pixel stored as an N x 3 array “mean_pixels”. You need to subtract each mean pixel from all pixels in the corresponding image in “images”. Which of the following lines of Python code achieves this?

A) images - mean_pixels[:, None, None]
B) images - mean_pixels[None, None]
C) images - mean_pixels[None, :, None]
D) images - mean_pixels

Correct: A)

For option A), mean_pixels[:, None, None] has shape (N, 1, 1, 3) so it will be broadcasted to chape (N, H, W, 3).

Q6

Consider the following output of the function level profiler cProfile after running a script that renders a movie of a simulated scene:

```
        1428 function calls (1427 primitive calls) in 17.101 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.000    0.000   17.101   17.101 {built-in method builtins.exec}
        1    0.003    0.003   17.101   17.101 dorender.py:1(<module>)
      201    0.001    0.000    8.841    0.044 render.py:13(render_scene)
      200    0.001    0.000    4.845    0.024 render.py:8(advance_scene)
        1    0.000    0.000    2.405    2.405 render.py:18(save_to_mp4)
        1    0.000    0.000    1.005    1.005 render.py:3(load_scene)
        1    0.000    0.000    0.002    0.002 <frozen importlib._bootstr...
        1    0.000    0.000    0.002    0.002 <frozen importlib._bootstr...
        1    0.000    0.000    0.002    0.002 <frozen importlib._bootstr...
```

What function takes the most time overall?

A) render_scene
B) advance_scene
C) save_to_mp4
D) load_scene

Correct: A)

The overall time is listed in the cumtime column. Of the listed options render_scene takes most time with 8.841 seconds.

Q7

The script which generated the profiling output from the previous question is shown below:

```
import sys
import render as R

scene_fname = sys.argv[1]
movie_fname = sys.argv[2]
scene = R.load_scene(scene_fname)

dt = 0.01  # Time step
n_steps = 200

all_scenes = [scene]
for i in range(n_steps):
    scene = R.advance_scene(scene, dt)
    all_scenes.append(scene)

all_frames = []
for scene in all_scenes:
    frame = R.render_scene(scene)
    all_frames.append(frame)

R.save_to_mp4(movie_fname, all_frames)
```

To speed-up the run time, we want to apply parallelization. Of the below options, what parts of the script are good candidates for parallelization?

A) The first for-loop which calls R.advance_scene
B) The second for-loop which calls R.render_scene
C) Both for-loops
D) None of the for-loops

Correct: B)

In the first for-loop each iteration depens on the previous since the scene variable is updated each time. Therefore, we cannot parallelize it.

In the second for-loop each iteration is independent so it is a good candidate for parallelization.

Q8

After adding parallelization to the Python program, an engineer measured the speed-up using 3 cores to be 2.5. Given this, what is the expected theoretical maximum speed-up?

A) Around 15x
B) Around 12x
C) Around 10x
D) Around 7x

Correct: C)

We solve the question using Amdahl's law. The theoretical speed-up for p cores is

$S(p) = 1/(1 – F + F/p).$

We now solve for F:

$1/S(p) = 1 – F + F/p,$ $F – F/p = 1 – 1/S(p),$ $F(p – 1)/p = 1 – 1/S(p)$

So,

$F = p(1 – 1/S(p))/(p – 1) = 3 * (1 – 0.4) / 2 = 0.9$

The theoretical maximum speed-up is then

$1/(1 – F) = 10$

Q9

Consider the following output from using the "time" command to time a Python program:

```
real    0m12.03s
user    0m12.00s
sys     0m0.034s
```

The program was run single threaded. Assume the program can be perfectly parallelized. We now run it on two cores and again use the "time" command to time it. What is the expected output?

A)
```
real    0m6.03s
user    0m6.00s
sys     0m0.034s
```

B)
```
real    0m6.03s
user    0m12.00s
sys     0m0.034s
```

C)
```
real    0m12.03s
user    0m6.00s
sys     0m0.034s
```

D)
```
real    0m12.03s
user    0m12.00s
sys     0m0.034s
```

Correct: B)

The first line of the time output (real) refers to the wall time and the next two (user and sys) refer to CPU time. For parallelization, we expect the wall time to go down, but the CPU time will remain constant (or even increase) since the CPU time is summed over each core.

Q10

Consider the following kernel summaries from two different Python programs profiled with nsys:

```
** CUDA GPU Kernel Summary (gpukernsum):

 Time (%)  Total Time (ms)  ... Avg (ms) ... StdDev (ms) ... Name
 --------  ---------------  ... -------- ... ----------- ... ----------
    100.0         500.0000  ...  20.0000 ...     40.0000 ... kernel1...
```

```
** CUDA GPU Kernel Summary (gpukernsum):

 Time (%)  Total Time (ms)  ... Avg (ms) ... StdDev (ms) ... Name
 --------  ---------------  ... -------- ... ----------- ... ----------
    100.0         500.0000  ...  20.0000 ...      0.0500 ... kernel2...
```

The programs were run on a computer with 4 GPUs meaning 4 kernels can run simultaneously. In both programs, work is divided to the GPUs based on static scheduling, i.e., tasks are divided equally up front, the kernels are all launched on the GPUs and we then wait for them to finish. Based on the profiling output, should another scheduling strategy be employed?

A) No, static scheduling is optimal for both cases.
B) Yes, the program with kernel1 should consider dynamic scheduling. The other program with kernel2 should stick with static scheduling.
C) Yes, the program with kernel2 should consider dynamic scheduling. The other program with kernel1 should stick with static scheduling.
D) Yes, both programs should use dynamic scheduling.

Correct: B)

Dynamic scheduling is beneficial when each task varies a lot in runtime. In such cases, dividing the work up front may result in one GPU having more slow tasks and the other GPUs will then finish first and then sit idle while waiting for the other GPU, i.e., we do not get the full benefit of having multiple GPUs. However, dynamic scheduling requires more overhead (since we need a task queue), so should only be used when needed.

For kernel1, there is a large standard deviation in the runtime, so it is a good candidate for dynamic scheduling. For kernel2, all kernels more or less the same time, so static scheduling is sufficient.

Q11

Consider the following output from the “kernprof” line profiler, which has profiled a function called “process”:

```
Timer unit: 1e-06 s

Total time: 3.27424 s
File: process.py
Function: process at line 97

Line #   Hits        Time  Per Hit   % Time  Line Contents
==============================================================
    97                                        @profile
    98                                        def process(data, params):
    99      1   2005064.0    2e+06     61.2      conds = prep_conds(params)
   100      1         4.0      4.0      0.0      result = []
   101   1001       740.0      0.7      0.0      for x in data:
   102   1000   1266748.0   1266.7     38.7          y = process_single(x, conds)
   103   1000      1685.0      1.7      0.1          result.append(y)
   104
   105      1         1.0      1.0      0.0      return result

  3.27 seconds - process.py:97 – process
```

The function processes data from the “data” argument according to a set of parameters in the “params” argument. For profiling, the “process” function was called with a small dataset. Normally, it would be called with a dataset of 10,000 entries. What is the estimated run time of “process” for a normal workload?

A) Around 14.7 seconds
B) Around 32.7 seconds
C) Around 3.5 hours
D) Around 12.7 seconds

Correct: A)

*The small dataset had 1000 elements since the for-loop in lines 101-103 ran 1000 times. For 10,000 this for-loop would take 10 times as long. The total time for the for-loop is (740 + 1266748 + 1685) * 10^{-6} = 1.269 seconds. For a normal workload it will then take 12.69 seconds. Since prep_conds does not depend on the "data" argument, it will have the same runtime, so the final runtime is 12.69 + 2.01 = 14.70 seconds.*

Q12

Consider the following CUDA kernel function which renders a depth map of a volume "vol", i.e., each pixel in the output "dmap" is the distance to the nearest solid object in the volume. We define any point in "vol" with a value under 0.5 to be empty space and over 0.5 to be solid. The rendering is done by starting at the location of the output pixel and then stepping through the volume along the direction "ray_step" until we hit something solid.

```
@cuda.jit
def render_depthmap(dmap, vol, u_step, v_step, ray_step, max_steps):
    j, i = cuda.grid(2)
    stepx, stepy, stepz = ray_step
    if i < dmap.shape[0] and j < dmap.shape[1]:
        # Get location of pixel in dmap
        x = i * u_step[0] + j * v_step[0]
        y = i * u_step[1] + j * v_step[1]
        z = i * u_step[2] + j * v_step[2]
        # Count steps until we hit something
        for s in range(max_steps):
            vi, vj, vk = int(x), int(y), int(z)
            if 0 <= vi < vol.shape[0] and \
               0 <= vj < vol.shape[1] and \
               0 <= vk < vol.shape[2]:
                if vol[vi, vj, vk] >= 0.5:
                    break  # We hit something!
            x += stepx
            y += stepy
            z += stepz
        dmap[i, j] = s
```

Assume we call the kernel with thread blocks of size 16 x 16. Assume also that dmap and vol are stored row-wise and that max_steps is 500. Which of the following values for u_step, v_step and ray_step arguments do we expect to have the **worst** performance with respect to memory efficiency?

A) u_step: [1.0, 0.0, 0.0]
v_step: [0.0, 1.0, 0.0]
ray_step: [0.0, 0.0, 1.0]

B) u_step: [0.0, 1.0, 0.0]
v_step: [0.0, 0.0, 1.0]
ray_step: [1.0, 0.0, 0.0]

C) u_step: [1.0, 0.0, 0.0]
v_step: [0.0, 0.0, 1.0]
ray_step: [0.0, 1.0, 0.0]

D) All inputs should have the roughly same performance.

Correct: A)

The threads of the kernel repeatedly reads from vol[vi, vj, vk] where vi, vj and vk changes every loop iteration according to ray_step. Ideally, threads with adjacent ID (which are in the same warp) should read from adjacent memory addresses. Since vol is stored row-wise, the last axis has the lowest stride. Therefore, good memory acess will happen for option B) and C) since v_step is [0, 0, 1]. For option A), v_step and u_step go along axes with larger strides, so we expect this to have the lowest performance w.r.t. memory efficiency.

Q13

Consider again the same CUDA kernel function as in the previous questions which renders a depth map of a volume:

```
@cuda.jit
def render_depthmap(dmap, vol, u_step, v_step, ray_step, max_steps):
    j, i = cuda.grid(2)
    stepx, stepy, stepz = ray_step
    if i < dmap.shape[0] and j < dmap.shape[1]:
        # Get location of pixel in dmap
        x = i * u_step[0] + j * v_step[0]
        y = i * u_step[1] + j * v_step[1]
        z = i * u_step[2] + j * v_step[2]
        # Count steps until we hit something
        for s in range(max_steps):
            vi, vj, vk = int(x), int(y), int(z)
            if 0 <= vi < vol.shape[0] and \
               0 <= vj < vol.shape[1] and \
               0 <= vk < vol.shape[2]:
```

```
            if vol[vi, vj, vk] >= 0.5:
                break  # We hit something!
        x += stepx
        y += stepy
        z += stepz
    dmap[i, j] = s
```

Given a volume of size 500 x 500 x 500, output image of size 200 x 200 and thread blocks of size 16 x 16, how many thread blocks are needed?

A) 13 x 13
B) 32 x 32 x 500
C) 32 x 32
D) 3 x 3

Correct: A)

Since each thread write one element of the out image, we only have to consider the size of dmap which is 200 x 200. Since 200 / 16 = 12.5 we need 13 x 13 thread blocks to cover the output image.

Q14

Consider the following output from the nsys profiler after running the “render_depthmap” kernel (some parts of the profiler output omitted):

```
** CUDA GPU Kernel Summary (gpukernsum):

 Time (%)  Total Time (ms)  Instances  Avg (ms) ...         Name
 --------  ---------------  ---------  -------- ... ----------------------...
    100.0          13.8403          1   13.8403 ... cudapy::__main__::rend...

 ** GPU MemOps Summary (by Time) (gpumemtimesum):

 Time (%)  Total Time (ms)  Count  Avg (ms) ...     Operation
 --------  ---------------  -----  -------- ... ------------------
     99.4          26.7968      4    6.6992 ... [CUDA memcpy HtoD]
      0.6           0.1608      4    0.0402 ... [CUDA memcpy DtoH]

 ** GPU MemOps Summary (by Size) (gpumemsizesum):

 Total (MB)  Count  Avg (MB)  Med (MB) ...     Operation
 ----------  -----  --------  -------- ... ------------------
    125.000      4    31.250     0.000 ... [CUDA memcpy HtoD]
      2.000      4     0.500     0.000 ... [CUDA memcpy DtoH]
```

Which of the following parts takes the most time?

A) Running the kernel
B) Host to device transfers (HtoD, CPU to GPU)
C) Device to host transfers (DtoH, GPU to CPU)
D) All parts take roughly the same amount of time

Correct: B)

We look at the "Total Time (ms)" column in each section. HtoD transfers take 26.8 ms which is more than the DtoH transfers (0.2 ms) and the kernel time (13.8 ms).

Q15

Consider the following Python program which defines a CUDA kernel and then calls it with NumPy arrays:

```
from numba import cuda
import numpy as np

@cuda.jit
def square(y, x):
    i = cuda.grid(1)
    y[i] = x[i] * x[i]

x = np.zeros(1024**3, dtype='uint8')
y =  np.empty_like(x)
square[1024*1024, 1024](y, x)
```

How many host to device (HtoD) and device to host (DtoH) memory transfers will this code perform? How many are necessary in an optimal implementation?

A) Will do 2 HtoD and 2 DtoH transfers, but only 1 HtoD and 1 DtoH are necessary.
B) Will do 1 HtoD and 1 DtoH transfers, and 1 HtoD and 1 DtoH are necessary.
C) Will do 2 HtoD and 2 DtoH transfers, and 2 HtoD and 2 DtoH are necessary.
D) Will do 2 HtoD and 1 DtoH transfers, and 1 HtoD and 2 DtoH are necessary.

Correct: A)

Since we call the kernel with NumPy arrays, Numba will transfer both x and y to the GPU and then back again. Therefore, the code will to 2 HtoD transfers and 2 DtoH transfers. However, since x is the input and y is the output, only 1 HtoD transfer and 1 DtoH transfer is necessary.

Q16

You find that one of your programs have low performance. You inspect the code, and you find that you access a large data structure in a random fashion. What is the likely cause of the low performance?

A) A slower part of the memory hierarchy is used.

B) The caches are too small.

C) There are too few cache levels.

D) There is not enough information to find the root cause.

Correct: A)

With random access it is unlikely that larger caches or more cache levels will improve performance.

Q17

Consider the following summary of a pandas DataFrame:

Col. name	dtype	#unique	min	max	example	size
filename	object	487 M	N/A	N/A	packinglist.docx	21.3 GB
version	int64	43	0	42	1	2.1 GB
sizediff_kb	int64	4366	-12	4353	203	2.1 GB

We wish to reduce the size of the DataFrame. How might we best reduce the size of the "version" column?

A) Convert to a datetime data type
B) Convert to a smaller floating point type
C) Convert to a smaller integer type
D) Cannot be reduced

Correct: C)

All values of the version column fit in an uint8 (min: 0, max: 255) or int8 (min: -128, max: 127) datatype. Since it is numeric data, we should not convert to datetime data types. Since it is integer data, we should not convert to a floating point data types as we may run into rounding errors when doing any further processing.

Q18

Consider the following summary of another pandas DataFrame:

Col. name	dtype	#unique	min	max	example	size
fname_hash	int64	487 M	17542445	91284623	41625192	2.1 GB
version	int64	612 M	0	42	1	2.1 GB

sizediff_kb	int64	4366	-12	4353	203	2.1 GB

We wish to run a processing script on this DataFrame on a computer system with only 24MB RAM. We will perform the processing in chunks of N rows at a time to reduce memory needs. Assume no memory reductions are made on the DataFrame. Of the following alternatives, what is the maximum chunk size we may use?

A) 800000
B) 2400000
C) 1100000
D) 500000

Correct: A)

*Each column uses 8 bytes so each row uses 8 * 3 = 24 bytes. 1 MB = 1000000 bytes, so 24 * 1000000 / 24 = 1000000, i.e., option A). We may also use 1 MB = 1024*1024 bytes, in which case 24 * 1024 * 1024 / 24 = 1048576, so option A) is still the only valid answer.*

Q19

Consider the below Python program:

```
import numpy as np

x = np.memmap('bigarray_u8.raw', mode='r',
              dtype='uint8', shape=10_000_000_000)
y = np.array(x[::100_000])  # Copy data to y
print(y.mean())
```

Disregarding the Python interpreter, what is the maximum memory requirement of the above Python program?

A) Around 10 GB
B) Around 100 MB
C) Around 10 KB
D) Around 100 KB

Correct: D)

The array in bigarray_u8.raw has 10,000,000,000 elements and we load every 100,000th element for a total of 10,000,000,000 / 100,000 = 100,000 elements. Each of these takes 1 byte, since the data type is uint8, so we use 100 KB.

Q20

Consider the following Python function:

```
def process(a):
    s = np.zeros(a.shape[0])
    for i in range(a.shape[0]):
        s[i] = np.sum(a[i])
    return s
```

Assume we call the "process" function with a Zarr 1024 x 1024 array of data type float64. What chunk size will result in the best performance?

A) (1, 1024)
B) (1024, 1)
C) (32, 32)
D) Chunk size will not affect performance

Correct: A)

Each iteration of the loop in process computes the sum over a row of "a". For option A), this requires loading 1 chunk per iteration, for option B) it requires loading 1024 chunks and for option C) it requires loading 1024 / 32 chunks. Therefore, option A) will have the best performance since it requires the lowest number of chunk loads.

Q21

A program simulates the motion of N objects in a solar system. A function calculates the gravitational force between all pairs of objects. The following numpy arrays are used:

- pos, shape (N, 3) where 3 is the number of spatial dimensions. Contains the positions of all objects for all timesteps.
- forces, shape (N, N, 3). Contains the force vector between all pairs of objects.
- mass, shape (N,). Contains the mass of all objects

The Python code is shown below, where numpy has been imported as np.

```
def calc_forces(pos, mass):

    N = pos.shape[0]

    G = 6.6743e-11 % Gravitational constant

    forces = np.zeros((N, N))

    for i in range(N):
```

```
        for j in range(N):
            p0 = pos[i], p1 = pos[j]
            m0 = mass[i], m1 = mass[j]
            r = np.sqrt(np.sum((p1 – p0) * (p1 – p0)))
            forces[i,j] = G * m0 * m1 / (r * r)
    return forces
```

The code is correct but slow. Which of the following statements is **not** correct?

A) Using the numba @jit decorator on the function is expected to improve performance
B) The loops can be easily parallelized
C) The processing time can be improved using only numpy
D) The order of the loops can be switched with no impact

Correct: D)

The forces array is stored and accessed row-wise with the shown order of the loops giving the benefit of spatial locality in the caches. If the order of the loops is swapped, this advantage is lost. Option A: Numba jit will compile the loops to machine code, which is expected to be faster than Python code. Option B: The loops can be easily parallelized since there are no dependencies between loop iterations. Option C: The loops could be replaced by operations on the numpy arrays directly, which is expected to be faster, since the numpy backend is efficient with large arrays.

Q22

A sports ball manufacturer performs computational fluid dynamics simulations of a ball's motion through the air for 10 newly proposed designs. For each design, a large range of parameters are simulated. The simulation state (ball geometry, speed, height, direction, etc.) is stored in a Python dictionary called params. In the following code, all_params contains a Python list with parameters for all simulations:

```
def simulate_ball(params):
    while params.height > 0:
        params = simulate_one_time_step(params)
    save_results(params)


def simulate(all_params):
```

```
for params in all_params:
    simulate_ball(params)
```

The function simulate_one_time_step is written in Python by another developer and may not be modified. Profiling on a small number of samples reveals run times of the simulate_ball function between a few minutes and a few hours. Given this information, which of the following approaches is expected to improve performance the most?

A) Use multithreading with dynamic scheduling to call simulate_one_time_step in parallel to simulate more time steps in parallel
B) Use multiprocessing with dynamic scheduling to run several calls to simulate_ball in parallel
C) Use multithreading with static scheduling to run several calls to simulate_ball in parallel
D) Use a GPU to run several calls to simulate_ball in parallel

Correct: B)

The calls to simulate_one_time_step are sequentially dependent eliminating option A. Different simulations take a different amount of time to complete, so static scheduling (option C) and GPU processing (option D) are bad choices. Additionally, the information that the simulate_one_time_step function is written in Python can be used to eliminate multithreading (options A and C) since the GIL will prevent improvements.

Q23

A colleague asks for your advice on speeding up a function. Your colleague is not allowed to show you the code but presents the following information. The function takes two arguments: a large numpy array of approximately 10 GB and the number of iterations to run. In each iteration, each array element is updated with no dependencies on other array elements and with a fixed number of floating-point operations. The function returns the updated array after the specified number of iterations. Performance benchmarking has been done using 5 and 10 iterations, but in real use millions of iterations should be run.

A heavily optimized CPU implementation was found to be too slow taking 0.5 seconds for 5 iterations and 1 second for 10 iterations. A simple GPU implementation was found to take 0.85 seconds for 5 iterations and 1.1 second for 10 iterations including transferring the input array to device memory and the output array back to host memory. Since benchmarking showed no improvement, it has been decided to not use the GPU implementation. Your colleague now asks for your advice. What do you say?

A) The described problem is not suited for GPU implementation

B) If the CPU implementation is already optimized, no further performance improvements can be expected
C) The simple GPU implementation should be optimized for them to expect any performance improvement
D) The current GPU implementation already has better performance, and they will see that by running more iterations

Correct: D)

The CPU implementation takes 0.1 second per iteration, while the GPU implementation takes (1.1-0.85)/(10-5)=0.25 seconds per iteration. The overhead of copying input and output is 0.6 seconds, but this is constant (independent of the number of iterations). The small number of iterations used in the benchmark led to the false conclusion. For option A, the described problem is trivially parallelizable and well-suited for GPU implementation. For option B, other hardware such as GPUs may vastly outperform even optimized CPU implementations. For option C, the basic GPU implementation already outperforms the CPU implementation when correctly taking the overhead of memory transfers into account.

Q24

Assume you have a program that runs in thirty-two (32) seconds on one processor. Your manager wants the program to run faster than eight (8) seconds. You profile the program and find one function that takes up sixteen (16) seconds of the execution time and can easily be made parallel. What should you tell your manager?

A) The program cannot run faster than sixteen (16) seconds.

B) The program can possibly be made to run faster than eight (8) seconds, but further analysis of the program is needed.

C) The program can be made to run faster than eight (8) seconds by rewriting the sequential part of the program.

D) The program cannot run faster than (16 + 16/p) seconds where p is the number of processors.

Correct: B)

We know that 16 of the 32 seconds can be easily parallelized. However, no information is available about the other 16 seconds, so it is not possible to say if the 8 second target can be reached.